

where $\mathcal{K} = \{\phi_1, \phi_2\}$ and the notation $dog \leftarrow \mathcal{B}_{dog}$ (similarly $cat \leftarrow \mathcal{B}_{cat}$) means variable dog is grounded with images from $\mathcal{B}_{dog}$, which is a batch of dog images randomly sampled from the set of dog images in the dataset. Figure 2 shows how LTNtorch builds the computational graph for this basic task.

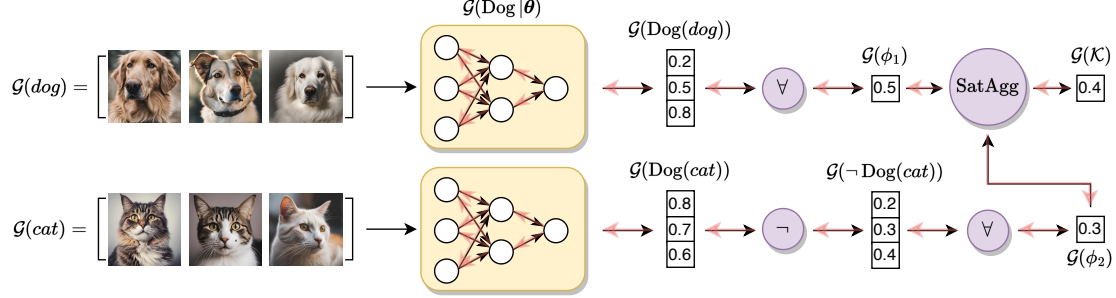


Figure 2: Computational graph of the LTN for binary classification. Batch size for $\mathcal{B}_{dog}$ and $\mathcal{B}_{cat}$ is 3. Red arrows indicate how gradients are back-propagated through the logical structure to change the weights of predicate Dog so $\mathcal{K}$ is maximally satisfied. LTNtorch minimizes $1. - \mathcal{G}(\mathcal{K}) \equiv \mathcal{L}(\theta)$.

3.1 LTNtorch implementation

In what follows, we provide a partial code snippet implementing the training loop of the presented example in LTNtorch. We invite the reader to review the examples in the repository and the framework documentation to complete this implementation. Note the implementation details of `CNN_model` and `train_data_loader` are not given as it is standard PyTorch code.

```
import ltn
# define the Dog predicate
Dog = ltn.Predicate(CNN_model())
# define logical operators and formula aggregator (i.e., SatAgg)
Not = ltn.Connector(ltn.fuzzy_ops.NotStandard())
Forall = ltn.Quantifier(ltn.fuzzy_ops.AggregPMeanError(p=2), quantifier="f")
SatAgg = ltn.fuzzy_ops.SatAgg()
# training loop
for epoch in range(n_epochs):
    train_loss = 0.0
    for i, (dog_imgs, cat_imgs) in enumerate(train_data_loader):
        optimizer.zero_grad()
        # ground logical variables with current training batch
        dog = ltn.Variable("dog", dog_imgs) # positive examples
        cats = ltn.Variable("cats", cat_imgs) # negative examples
        # compute loss function
        sat_agg = SatAgg(
            Forall(dogs, Dog(dogs)), # this is phi_1
            Forall(cats, Not(Dog(cats))) # this is phi_2
        )
        loss = 1. - sat_agg
        # back-propagation
        loss.backward()
        optimizer.step()
        train_loss += loss.item()
    train_loss = train_loss / len(train_data_loader)
```

Listing 1: LTNtorch implementation of the dog classification example.

4 Conclusions

In this paper, we introduced LTNtorch, the fully documented and tested PyTorch implementation of Logic Tensor Networks. We summarized the formalization of the framework and provided a simple example to help understand how LTNtorch implements the LTN. This paper does not have to be considered a comprehensive framework documentation. We strongly invite practitioners interested in approaching LTNs to read the original paper and the documentation carefully.

References

- Samy Badreddine, Artur d’Avila Garcez, Luciano Serafini, and Michael Spranger. Logic tensor networks. *Artificial Intelligence*, 303:103649, 2022. ISSN 0004-3702. doi: <https://doi.org/10.1016/j.artint.2021.103649>. URL <https://www.sciencedirect.com/science/article/pii/S0004370221002009>.
- Petr Cintula, Petr Hájek, Carles Noguera, et al. *Handbook of Mathematical Fuzzy Logic. Volume 1*, volume 37. College Publications, 2011.
- Artur S. d’Avila Garcez, Luís C. Lamb, and Dov M. Gabbay. *Neural-Symbolic Learning Systems*, pages 35–54. Springer Berlin Heidelberg, Berlin, Heidelberg, 2009. ISBN 978-3-540-73246-4. URL https://doi.org/10.1007/978-3-540-73246-4_4.
- Peter A. Flach. *First-Order Logic*, pages 35–68. Springer Berlin Heidelberg, Berlin, Heidelberg, 2007. ISBN 978-3-540-74113-8. doi: 10.1007/978-3-540-74113-8_2. URL https://doi.org/10.1007/978-3-540-74113-8_2.
- Emile van Krieken, Erman Acar, and Frank van Harmelen. Analyzing differentiable fuzzy logic operators. *Artificial Intelligence*, 302:103602, 2022. ISSN 0004-3702. doi: <https://doi.org/10.1016/j.artint.2021.103602>. URL <https://www.sciencedirect.com/science/article/pii/S0004370221001533>.